INDEX

NAME: L. Srimadura STD.: SEC.: ROLL NO.: SUB.:

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1	29/7/26	Solve the 8 queen problem		
2	5/8/26	Solve a graph using DFS & BFS		
3	5/8/26	Implement minmax algorithm		
4	5/8/26	Implement A* algorithm		
5	12/8/26	~~Programs on~~ Decision making and ~~Knowledge representation~~ Intro to Prolog		
6	13/8/26	Unification & Resolution		
7	18/8/26	Backward chaining		
8	19 23/8/26	Forward chaining		
9	30/8/26	Implementation of Block world program		
10	16/9/26	Implementation of A Fuzzy Inference system		
11	23-09-26	Implementing Artificial neural Neyworks for an Application using python		

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
12	30-9-26	Implementation of Decision Tree		
13	7-10-26	Implementation of K-mean algorithm		

Ex. No. 1
Date : 22/7/26

Programs on Problem Solving

8 - QUEENS PROBLEM

a)

Aim :

Write a program to solve 8 - queens problem program

Code :

```
def solve_n_queens(n):
    board = [-1] * n

def is_safe(row,col):
    for prev_row in range(row):
        prev_col = board[prev_row]

        if prev_col == col:
            return False

        if abs(prev_col - col) == abs(prev_row - row):
            return False

    return True

def backtrack(row):
    if row == n:
        print_solution(board)
        return True

    for col in range(n):
        if is_safe(row, col):
            board[row] = col
            if backtrack(row+1):
                return True
            board[row] = -1

return False
```

```
    def print_solution(board):
        for row in range(n):
            for col in range(n):
                if board[row] == col:
                    print("Q", end=" ")
                else:
                    print(".", end=" ")
            print()

    backtrack(0)

Solve_n-queens(8)
```

Output:

```
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Exp - 2 Breadth - First Search BFS
Date - 29/7/26 Depth - First search DFS

Depth - First search

Q. Aim :

Solve any problem using depth first Search

Code :

```
def dfs (graph, node, visited):
    if node in visited :
        return

    visited add (node)
    print (node)

    for neighbour in graph[node] :
        dfs (graph, neighbor, visited)


graph = {
        'A' : ['B', 'C'],
        'B' : ['D', 'E'],
        'C' : ['F'],
        'D' : [],
        'E' : ['F'],
        'F' : []
}

visited = set()
dfs (graph, 'A', visited)
```

Output :

```
A
B
D
E
F
C
```

b)

Breadth first search

AIM :

Solve any problem using breadth-first search (BFS)

Code :

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    visited.add(start)

    while queue:
        node = queue.popleft()
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

bfs(graph, 'A')
```

output :

A B C D E F

Exp-5
Date-2/9/26

minmax algorithm implementation

Aim:
To implement the minimax algorithm in python for finding the optimal move in a two-player game.

Code:

```
def minmax(depth, node_index, maximizing_player,
                                values, height):
    if depth == height:
        return values[node_index]
    if maximizing_player:
        return max(
            minmax(depth+1, node_index*2, False,
                                values, height)
        )
    else:
        return min(
            minmax(depth+1, node_index*2, False,
                                values, height),
            minmax(depth+1, node_index*2+1, True,
                                values, height)
        )
values = [3, 5, 2, 9, 12, 5, 23, 23]
height = 3
result = minmax(0, 0, True, values, height)
Print("Optimal value:", result)
```

Output:

Optimal value : 12

Result: The algorithm has been implemented and executed and

Exp-5
Date - 5/8/26

Introduction to Prolog

AIm: Introduction to prolog

Program:

intro.pl

```
% Facts
parent (ram, sita)
parent (ram, ravi)
parent (sita, anu)

% Rule
grandparent (X, Y) :-
    parent (X, Z)
    parent (Z, Y)
```

Result:

```
?- parent(ram, sita)
true
?- parent (ram, ravi)
true
?- grandparent (ram, anu)
true
```

Exp 14
Date 29/09/25

Implementation of A* algorithm

Aim:

To implement A* algorithm

Code:

```
def a_star(graph, h, start, goal):
    open_list = [(h[start], start)]
    g = {start: 0}
    parent = {}
    while open_list:
        open_list.sort()
        f, node = open_list.pop(0)

        if node == goal:
            path = []
            while node in parent:
                path.append(node)
                node = parent[node]
            path.append(start)
            print("Path", path[::-1])
            print("Cost", g[goal])
            return

        for neighbor, cost in graph[node]:
            new_cost = g[node] + cost
            if neighbor not in g or new_cost < g[neighbor]:
                g[neighbor] = new_cost
                parent[neighbor] = node
                open_list.append((new_cost + h[neighbor],
                                  neighbor))

        print("No path found")

graph = {
    'A': [('B',1), ('C', 3)],
    'B': [('D',3), ('E', 1)],
    'C': [('F', 5)],
    'D': [('G', 2)],
    'E': [('G', 5)],
    'F': [('G', 2)],
    'G': []
}
```

h = {'A': 5, 'B': 4, 'C': 4, 'D': 2, 'E': 4, 'F': 2, 'G

astar_graph, h, 'A', 'G')

S
Li
as
w

1.

2.

3.

4.

5.

6.

7.

8.

9.

If y
If n

Result:

Path: ['A', 'B', 'D', 'G']

Cost: 6

Fin
Mor

Exp - 6
Date - 5/8/26

Implementation of unification algorithm and Resolution

Aim: To implement unification and resolution algorithm

Program:

```
def unify(x, y, subs={}):
    if x == y:
        return subs
    if isinstance(x, str) and
x.islower():
        subs[x] = y
        return subs
    if isinstance(y, str) and
y.islower():
        subs[y] = x
        return subs
    if isinstance(x, list) and
isinstance(y, list) and len(x) == len(y):
        for a, b in zip(x, y):
            subs = unify(a, b, subs)
        return subs
    return None
x = ['p', 'x', 'y']
y = ['p', 'John', 'Mary']
print("Unification:", unify(x, y, {}))


def Resolve(c1, c2):
    for x in c1:
        for y in c2:
            if x == '!'+y or y == '!'+x:
            return (c1 - {x}) | (c2 - {y})
            return
    return None
c1 = {'P', 'Q'}
c2 = {'!P', 'R'}

result = resolve(c1, c2)
print("Resolvent:", result)
```

Result:

Resolvent: {'Q', 'R'}

Exp - 7
Date - 23/8/26

Implementation of Backward chaining Algorithm

Aim:

Program:

```
facts = {'A', 'B'}

rules = {
    'C': ['A', 'B'],
    'D': ['C'],
    'E': ['B', 'C']
}

def backward_chaining(goal):
    print("Checking goal", goal)
    if goal in facts:
        print(goal, "is a known fact")
        return True
    if goal in rules:
        print("Applying rule for", goal)
        for condition in rules[goal]:
            if not backward_chaining(condition):
                return False

        print(goal, "is proved")
        return True

goal = input("Enter the goal: ")
if backward_chaining(goal):
    print("\nResult: Goal proved")
else:
    print("\nResult: Goal cannot be proved")
```

Result :

Enter the goal : D
Checking goal : D
Applying rule for : D
Checking goal : C
Applying rule for : C
Checking goal : A
A is a known fact
Checking goal : B
B is a known fact
C is proved
D is proved

Exp - 8
Date - 19/8/25

Implementation of Forward Chaining algorithm

Program:

```
facts = {'A', 'B'}
rules = {
    'C' : ['A', 'B'],
    'D' : ['C'],
    'E' : ['B', 'C']
}

def forward_chaining(facts, rules):
    inferred = set(facts)
    changed = True
    while changed:
        changed = False
        for conclusion, conditions in rules.items():
            if all(condition in inferred for
                   condition in conditions):
                if conclusion not in inferred:
                    inferred.add(conclusion)
                    print("Derived:", conclusion)
                    changed = True

    return inferred

print("Initial Facts:", facts)
result = forward_chaining(facts, rules)
print("Final Facts:", sorted(result))
goal = input("Enter the goal:").strip()
if goal in result:
    print("Goal proved")
else:
    print("Goal not proved")
```

Result:

Initial facts : {'A', 'B'}

Derived : C

Derived : D

Derived : E

Final facts : ['A', 'B', 'C', 'D', 'E']

Goal proved

Implementation of blocks World program

Program:

```
Initial_State = ['A', 'B', 'C']
Goal_State = ['C', 'B', 'A']
print ("Initial State:", Initial_State)
print ("Goal State:", Goal_State)

def block_world (initial, goal):
    State = initial.copy()
    print ("Insteps to reach the goal:")
    for i in range (len(goal)):
        if state[i] != goal[i]:
            j = state.index (goal[i])
            state[i], state[j] = State[j], State[i]
            print ("Move", goal[i], "to position", i+1)
            print ("Current State:", State)
    print ("Final State:", State)

    if State == goal:
        print ("Goal State reached successfully!")
    else:
        print ("Goal State not reached.")

block_world (Initial_State, Goal_State)
```

Result:

Initial State: ['A', 'B', 'C']
Goal State : ['C', 'B', 'A']

Steps to reach the goal:
move C to position 1
Current State: ['C', 'B', 'A']
Final State: ['C', 'B', 'A']
Goal State reached successfully

Exp - 10
Date - 16/9/20

Implementation of Fuzzy Influence System

Aim:

Program:

```
def fuzzy_intervence(temperature):
    if temperature <= 20:
        cold = 1
    elif temperature < 30:
        cold = (30 - temperature)/10
    else:
        cold = 0

    if temperature <= 20 or temperature >= 40:
        warm = 0
    elif temperature <= 30:
        warm = (temperature - 20)/10
    else:
        warm = (40 - temperature)/10

    if temperature <= 30:
        hot = 0
    elif temperature < 40:
        hot = (temperature - 30)/10
    else:
        hot = 1

    print("Temperature:", temperature)
    print("Cold:", round(cold, 2))
    print("Warm:", round(warm, 2))
    print("Hot:", round(hot, 2))

    if hot > warm and hot > cold:
        print("Result : Fan Speed HIGH")
    elif warm >= cold:
        print("Result : Fan Speed MEDIUM")
    else:
        print("Result : FAN Speed LOW")
```

fuzzy - inference(35)

Result:

Temperature: 35
Cold : 0
Warm: 0.5
Hot : 0.5
Result: Fan Speed MEDIUM

Exp : 11
Date : 23-9-26

Implementing Artificial Neural Neyworks for an Application using python

Aim:

To implement artificial Neural Neyworks for an application using python

Program :

```
import numpy as np
from sklearn.neural_network import MLPClassifier

x = np.array([[0,0],[0,1],[1,0],[1,1]])

model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='relu',
    solver='lbfgs',
    max_iter=5000,
    random_state=42
)
model.fit(x, y)

print("Predictions:")
for i in x:
    print(i, "->", model.predict([i])[0])
```

Result:

Predictions:

[0 0] -> 0
[0 1] -> 1
[1 0] -> 1
[1 1] -> 0

Exp: 12
Date: 30-9-26

Implementation of Decision tree

AIm:

To implement Decision tree using python

Program:

```
from sklearn.tree import DecisionTreeClassifier
X = [[1,1],[1,0],[0,1],[0,0]]
y = [1,1,0,0]

model : DecisionTreeClassifier()
model.fit(X,y)

print("Prediction:", model.predict([[1,1]]))
print("Prediction:", model.predict([[0,1]]))
```

Result:

```
Prediction:[1]
Prediction :[0]
```

EXP - 13
Date : 7-10-26

Implemention of K-mean algorithm

Aim:

To implement K-means Algorithm using Python

Program:

```
from sklearn.cluster import KMeans
X = [[1,2], [2,3], [3,4], [8,9], [9,10], [10,11]]
model = KMeans(n_clusters = 2, random_state = 0
n_init = 10)
model.fit(X)
print("cluster Labels:", model.labels_)
print("Centroids:", model.cluster_centers_)
```

Result:

```
Cluster Labels = [1 1 1 0 0 0]
Centroids : [[9. 10.]
 [2. 3.]]
```